# 刚装完 Ubuntu，接下来做什么

**这本手册给谁看**：刚把 Ubuntu 装好、准备开始打 ICUAS 2027 无人机竞速的人。
**它解决什么**：从"一个空系统"走到"能在仿真里看到无人机"。
**它不解决什么**：不教你写代码，也不教飞控理论——那些在 `docs/01` 和 `docs/06` 里。

读法：按顺序做，每步都有"你会看到什么"。**看不到就不要往下走**，翻到最后的问题表。

---

## 开始前，先确认三件事

| 要确认的 | 怎么确认 | 不够会怎样 |
|---|---|---|
| 磁盘至少剩 **40 GB** | `df -h /` | 构建到一半空间不足，白等 40 分钟 |
| Docker 能用 | `docker run --rm hello-world` | 整条路走不通（官方栈只能跑在容器里） |
| 有显示器、能开图形界面 | 桌面能打开终端 | 最后一步只能看数据、看不到画面 |

**为什么非要 Docker?** 官方的 LASER UAV System 只支持 **Ubuntu 22.04 + ROS 2 Humble**。如果你装的是 24.04（现在默认就是这个），直接在系统里装 Humble 会把整个系统搞乱。Docker 里跑一个 22.04 环境，是最省事的办法。

---

## 第 1 步：把项目拿下来（1 分钟）

```
cd ~
git clone git@github.com:E-Goose-Jerry/icuas27-racing.git uav-racing
cd uav-racing
./tools/preflight.sh
```

**你会看到**：一串 `[ok]`，最后一行 `all clear`。

这一步其实是个体检：课程文件能不能解析、149 个单元测试过不过、Docker 和显示器在不在。`[fail]` 必须先解决；`[warn]` 可以先记着。

> 顺便说一句，这里跑的 149 个测试**不需要 ROS、不需要 Docker、不需要无人机**。
> 它们是纯 Python 的竞赛逻辑（过门判定、计时、罚时、状态机），
> 这就是为什么它们能在任何一台电脑上验证。

## 第 2 步：构建仿真镜像（45–60 分钟，全程不用管）

```
./sim/run_sim.sh build
```

**这一步在做什么**：下载 Ubuntu 22.04 的底包，装 ROS 2 Humble、Gazebo、
编译 PX4 飞控固件、装 DDS 通信代理、编译 NMPC 求解器、再编译 35 个 ROS 包。
听起来多，但它只需要做一次。

**你会看到**：大量滚动日志，然后停下。中间有几段"什么都不输出"几分钟——
尤其 PX4 编译那段，那是在编译飞控固件，正常。

想随时看进度：

```
tail -f /tmp/icuas27_build.log
```

**怎么判断成功了**：

```
docker images | grep icuas27
```

看到 `icuas27-racing:local 19.8GB` 就成了。

> ⚠️ **只看命令有没有结束是不够的**。构建失败时命令也会"正常结束"，
> 只是没生成镜像。我自己在这上面栽过好几次，最长一次白等 25 分钟。

**失败了怎么办**：翻到最后的「遇到问题怎么办」，按报错关键词对号入座。
每个坑都记在 `docs/07-环境与仿真排障日志.md` 里，带着根因和验证方法。

---

## 第 3 步：编译竞赛包（20 秒）

```
./sim/run_sim.sh build-race
```

**你会看到**：`Summary: 2 packages finished`

这两个包就是"竞赛层"：一个是消息/服务定义，一个是任务节点（门检测 + 状态机）。
飞行控制、轨迹规划那些**不用你编译**——那是官方维护的，镜像里已经有了。

---

## 第 4 步：冒烟测试 —— 这就是"搭好了"的判据

```
./sim/smoke_test.sh
```

它会用**不开窗口**的方式把整套东西拉起来，然后逐项检查。**你会看到 10 个 `[ok]`**：

```
[ok] icuas27_racing built
[ok] laser_uav_managers built
[ok] gzserver running                    ← 物理引擎在跑
[ok] PX4 SITL process running            ← 飞控固件在跑
[ok] MicroXRCEAgent running              ← 飞控和 ROS 的通信桥
[ok] topic /uav1/hw_api/odometry
[ok] topic /uav1/estimation_manager/estimation
[ok] topic /uav1/control_manager/state
[ok] topic /uav1/planning_manager/diagnostics
[ok] odometry: average rate: 100.4       ← 状态估计在 100 Hz 出数据
[ok] race_control service advertised     ← 我的任务节点在线
[ok] status document written
```

**为什么"里程计 100 Hz"这一项最重要**：它证明整条链路是通的——
物理引擎产生数据 → 飞控收到传感器 → 通过 DDS 送到 ROS → 状态估计器算出位姿。
任何一环断了，这一项就没有数字。而"话题存不存在"是骗人的，话题可能只是个空的订阅。

看到全绿，环境就算搭好了。**这一步不需要屏幕，也不会有东西飞起来。**

---

## 第 5 步：开窗口看一眼（需要显示器）

```
./sim/smoke_test.sh --visual
```

会弹出 Gazebo 窗口，占满屏幕。**你会看到**：一个约 5×5 米的灰色场地，
上面立着 **7 个门框**——那就是规则书里那 7 个 1 米×1 米的障碍，
无人机停在场地中央的地面上（还没解锁）。

这一层比前一步多踩了三个坑：X 授权、显卡设备、Gazebo 的 shader 路径。
都已经修好并记在 `docs/07` §12 里。

看完怎么关：

```
docker rm -f smoke
```

## 第 6 步：让它起飞（谨慎）

前五步都是"验证环境"。这一步是第一次真的动。

```
docker exec smoke bash -c '
  set +u; source /opt/ros/humble/setup.bash
  source /opt/laser_uav_system_ws/install/setup.bash; set -u
  ros2 service call /icuas27/race_control icuas27_interfaces/srv/RaceControl \
    "{command: start, reason: operator}"'
```

**会发生什么**：解锁电机 → 起飞到 1 米 → 开始计时 → 飞向 1 号门。

**第一次跑之前请确认**：窗口能看见、冒烟测试全绿、旁边有人。
可能会翻、可能会撞——那就正好说明哪里还需要修。

---

## 遇到问题怎么办

先按**报错里的关键词**查这张表。每一条的完整说明（现象/根因/修法/验证）
都在 `docs/07-环境与仿真排障日志.md`。

| 你会看到 | 一句话解释 | 查 |
|---|---|---|
| `Unable to locate package gazebo11` | 22.04 上这两个包叫 `gazebo` / `libgazebo-dev` | 07 §1 |
| `... provided by "ament_cmake" ... not find one` | 容器里 `source ~/.bashrc` 是无效操作，ROS 根本没生效 | 07 §2 |
| `AMENT_TRACE_SETUP_FILES: unbound variable` | 脚本开了 `set -u`，而 ROS 的 setup 脚本不吃这一套 | 07 §3 |
| `Could not find LIVOX_LIDAR_SDK_LIBRARY` | 分支选错，走进了"真机模式"（会多装 18 个驱动、且不编 PX4） | 07 §4 |
| 测试全绿但**没有里程计** | Gazebo 插件路径没设，飞控收不到传感器——而且**不报错** | 07 §5 |
| `no matching password record` | 容器里以宿主机 UID 跑，PX4 查不到用户 | 07 §6 |
| `Authorization required` | 要挂 `~/.Xauthority`，光有 X socket 不够 | 07 §12.1 |
| `Failed to query drm device` | 容器里没显卡设备，加 `--device /dev/dri` | 07 §12.2 |
| `Unable to find shader lib` | `GAZEBO_RESOURCE_PATH` 没设，gzclient 会崩（gzserver 不受影响） | 07 §12.3 |
| `No space left on device` | 磁盘满了，先 `docker system df` 看能回收多少 | 07 §9 |

如果都没有，按这个顺序找：
`docs/07`（环境）→ `docs/05`（逻辑）→ `AGENTS.md`（当前状态和"下一步"）。

---

## 日常怎么用

| 想做的事 | 命令 |
|---|---|
| 改赛道 / 换门位置 | 改 `course/*.json`，不用改代码 |
| 改航点速度、重试策略 | `ros2_ws/.../race_node.py` 的参数，或 launch 里的 `default_speed` / `retry_limit` |
| 看实时数据 | `python3 -m ground_station.server`（不需要 ROS，用回放模式） |
| 跑测试 | `python3 -m unittest discover -s tests -t .` |
| 重新生成赛道文件 | `python3 tools/make_courses.py` |
| 起飞前自检 | `./tools/preflight.sh` |

---

## 几个名词，免得看晕

| 名字 | 是什么 | 为什么要管它 |
|---|---|---|
| **LUS** | LASER UAV System，主办方给的整套飞行软件 | 飞行控制、规划都靠它，我们只写"竞赛层" |
| **PX4** | 飞控固件（真实硬件上也跑这个） | 它接收指令、输出电机转速 |
| **uXRCE-DDS** | 飞控和 ROS 之间的通信桥 | 桥断了，ROS 就看不到飞机 |
| **Gazebo** | 物理仿真器 | 提供"会掉下去的物理"和相机画面 |
| **里程计** | 飞机的当前位置和速度估计 | 我们判断"有没有过门"全靠它 |
| **acados** | NMPC 求解器 | 官方控制器要用它算最优输入 |
| **T_adj** | 调整后用时 = 净用时 + 罚时 | 规则书的排名依据，每次罚 5 秒 |

---

## 一句话总结

**前四步（体检 → 镜像 → 编译 → 冒烟）是"把地基铺平"，做完之后每次改动只需要几十秒。第五、六步才是"开始玩"。**

所有踩过的坑都留在 `docs/07` 里了。下次换机器、或者队友要搭一套，
照着这本手册走一遍就行——不用再花一晚上。

---